

Fuzzing libpng-1.2.56 with AFL++: a Heuristic-Learning Augmented Campaign

Group 44

EPFL CS-412 — Software Security

Abstract

We fuzz libpng-1.2.56 with AFL++ in two complementary modes (compile-time instrumentation with ASan/UBSan, and binary-only QEMU mode), characterise saturation, triage the resulting crashes against known CVEs, and contrast a baseline campaign against a small *heuristic-learning* (HL) loop that uses a coding-agent feedback round to evolve the harness, dictionary, and (optionally) a custom mutator between two 15-minute rounds.

1 Q1 — Harness design

Entry point. We chose the canonical pull-mode read pipeline: `png_create_read_struct` → `png_read_info` → a chain of `png_set_*` transformations → `png_read_update_info` → `png_read_image` → `png_read_end`. This is the path that real applications (browsers, ImageMagick, terminal image renderers) take, and it is the region where libpng’s historical CVEs cluster (PLTE expansion: CVE-2015-8126; tEXt parsing: CVE-2016-10087).

Alternatives considered. (i) A shallow harness that stops at `png_read_info` skips the IDAT defilter pipeline entirely, missing the IDAT-region CVEs. (ii) A `png_set_PLTE`-only micro-harness is deeper but narrower; it trades coverage for focus and is appropriate as a sibling, not a primary. (iii) The encoder API (`png_write_*`) is rarely attacker-controlled. (iv) Progressive-read (`png_push_*`) has its own state machine and warrants a separate harness.

Data flow and guards. See Listing A.1 in the appendix for the harness source. Each guard is justified: the `setjmp/png_jmpbuf` pair is required because libpng reports errors via `longjmp`; without it every malformed input becomes a SIGABRT and floods the campaign with false positives. The `MAX_PIXELS` cap (4096^2) refuses crafted oversize IHDRs that OOM the fork worker without exposing memory-safety bugs. The `MAX_INPUT_SIZE` cap (1 MiB) prevents zlib decompression blowup. The signature pre-check skips inputs that libpng will reject anyway, saving fork-server cycles.

2 Q2 — Instrumentation & sanitizers

We compile libpng twice: an instrumented build with `afl-clang-fast + AFL_USE_ASAN=1 + AFL_USE_UBSAN=1` at `-O1 -g -fno-omit-frame-pointer`, and a vanilla gcc build for the QEMU campaign. The single patch we apply is AFL++’s `libpng-nocrc.patch`, which rewrites `png_crc_finish` to unconditionally return 0. Without this patch, the overwhelming majority of mutated inputs would be rejected at the CRC check, never reaching the chunk data parser; with it, we trade the ability to fuzz a closed-source CRC-aware target for a $\sim 10\times$ effective coverage rate (PDF §6.2 acknowledges the trade). Detailed compiler-flag rationale is in Appendix A.2.

3 Q3 — Seeds and dictionary

Formal view. Treating PNG as the grammar $\text{PNG} ::= \text{SIG IHDR (PLTE)? IDAT}^+ \text{ANC}^* \text{IEND}$, the dictionary entries are exactly the *terminal symbols* (signature, chunk type tags, zlib stream prefixes, filter codes, IHDR field constants). AFL++’s `havoc/splice` mutators are byte-level approximations of context-free editing; the dictionary lifts the search to token level, raising the expected hit rate on a 4-byte chunk tag from $1/2^{32}$ per position to ~ 1 per few-dozen candidate insertions.

Seeds. Eight hand-picked PNGs (gray-8, RGB-8, palette, RGB+alpha, gray+alpha, Adam7-interlaced, text-chunk, gray-16) generated by `scripts/gen-seeds.py` and minimised with `afl-cmin`.

Quantitative dictionary contribution. Classifying the 498 discovered queue entries by the AFL++ `op: tag` (read off the queue file names): `havoc=195` (39.2%), `ext_U0=77` (15.5%), `quick=57` (11.4%), `splice=51` (10.2%), `flip1=41` (8.2%), `arith8=31` (6.2%), and smaller contributions from `int8/16/32`, `flip2/4/8/16`, and `arith16`. The `ext_U0` class is AFL++’s user-extra dictionary operation, so $\sim 15\%$ of new edges in this 31-minute run are directly attributable to the dictionary. The dominance of `havoc` (39%) is typical: `havoc` combines all mutators

including dictionary insertions, so the true dictionary share is higher than 15 %.

4 Q4 — Campaign analysis

31 minutes (1862 s) of fork-mode fuzzing on the instrumented target produced **807 edges** (of 3320 total instrumented edges in the binary, i.e. **24.31 % map density**), **506 corpus entries**, **100.00 % stability**, and **0 saved crashes**. The fuzzer performed 211919 executions at **113.8 exec/s** average and did not complete a single cycle (the corpus grew faster than it could be re-walked). The edge curve (Appendix Fig. A.4) shows: 546 edges at $t=61$ s, 720 at $t=300$ s, 764 at $t\approx 900$ s, and 807 at $t\approx 1800$ s — a steep early rise followed by stepped growth and a flat final minute. With `cycles_done=0` the campaign is *not* saturated by the strict definition, but the discovery rate has collapsed — a textbook soft-plateau. *Saturation is not the absence of bugs*: it is the exhaustion of this particular harness–corpus–dictionary configuration. The HL Round 2 (§9) uses exactly this signal as the trigger to evolve the harness.

5 Q5 — Crash triage

We did not observe any libpng-attributable crashes in either main campaign. We applied `patches/synthetic-bug.patch` (conditional `malloc(rowbytes-1)` per row when $w \cdot h > 300$ pixels — the guard keeps all seeds passing so the fork server can calibrate normally) and rebuilt as `png_fuzz_syn`. A 60-second fuzz campaign on the patched binary saved **2 crashes**: the first within 841 ms (at execution #177, via the quick havoc-style mutator that bumped an IHDR dimension above the threshold); the second at execution #6956 via the `ext_UO` dictionary-insertion mutator. `afl-tmin` reduced the 74-byte first crash to **58 bytes** (21.62 % reduction, 227 execs). ASan classifies the bug as a **heap-buffer-overflow** of 32-byte width 0 bytes past a 31-byte allocation (the off-by-one we injected). The symbolised stack (Appendix Listing A.5) is:

```
#0 __interceptor_memcpy
#1 png_combine_row in pngutil.c
#2 png_read_row at pngread.c:822
#3 png_read_image at pngread.c:940
#4 main at harness.c:176
allocated by main at harness.c:167
```

This proves the toolchain (compile-time instrumentation + AFL++ fork server + ASan + `afl-tmin`) is wired correctly end-to-end. We attribute the no-real-crash result on libpng-1.2.56 to: (i) it is the most-fuzzed version of the most-fuzzed PNG library, so easy bugs are gone; (ii) our harness exercises only the pull-mode read pipeline (Q6 details the gaps);

(iii) 31 min is far below the OSS-Fuzz day-scale budget. Q6’s missing surface and §9’s HL loop directly address (ii).

6 Q6 — Attack surface

Real-world consumers. (1) Chrome and Firefox image decoders process arbitrary network-supplied PNGs; a libpng heap-overflow becomes the head of a sandbox-escape chain. (2) ImageMagick, often invoked at root or service privilege for thumbnail generation, dispatches PNG decoding to libpng; a malicious upload plus a libpng RCE-class bug yields server-side RCE.

Code paths our harness misses. (a) ImageMagick’s `write/convert` path in `coders/png.c` eventually drives `png_write_*` in `pngwrite.c`; a malicious upload that is decoded and re-encoded can hit encoder-side bugs our read harness cannot see. (b) Browser streaming decoders feed partial network buffers through libpng’s progressive API (`png_process_data/png_push_*` in `pngpread.c`); our file-at-once pull harness only reaches `png_read_image`. (c) Applications may install private-chunk callbacks via `png_set_read_user_chunk_fn`, an extension surface absent from our harness. These gaps matter because they are different state machines, not just different inputs.

7 Q7 — Binary-only QEMU mode

A second build using stock `gcc` (no instrumentation, no sanitizer; CRC patch retained for comparability) was fuzzed for 31 min under `afl-fuzz -Q`. Side-by-side numbers:

	Instrumented + ASan	QEMU (vanilla)
exec/sec	113.80	71.52
edges found	807	2115
corpus count	506	353
bitmap cvg	24.31 %	3.23 %

Note on the edge counts: the raw `edges_found` numbers are *not* directly comparable across modes. Compile-time `afl-clang-fast` instruments at LLVM-IR conditional-branch granularity and bins into a 2^{16} -entry edge map of which only 3320 slots are live; QEMU mode hashes every translated basic-block transition into the full 2^{16} map, so its denominator is much larger and its semantics finer. Hence QEMU reports more raw edges (2115) but lower map density (3.23 % vs. 24.31 %). The `corpus_count` and `exec/sec` numbers are apples-to-apples.

Why QEMU is slower. QEMU user-mode emulation translates each basic block via TCG on first encounter and adds an instrumentation callback at every BB boundary; the callback hashes the `prev_loc` $\rightarrow$ `cur_loc` pair into AFL++’s edge map. Compile-time instrumentation knows the CFG statically

and inlines the counter increment, so QEMU pays both a translation tax (per BB) and a per-instruction emulation tax. Edge IDs in QEMU mode are address-hash-derived, so multiple source-level branches that compile to the same BB become a single QEMU edge — an additional source of coverage coarsening.

8 Q8 — Instrumentation depth and performance

Edge counts. A whole-library probe (`src/edge_probe.c` linked with `-Wl,-whole-archive libpng12.a`) reports **4788 total instrumented edges** at afl-fuzz startup. The final harness binary reports **3320 total instrumented edges** (`total_edges` in `fuzzer_stats`). The harness number is smaller even after adding harness code because normal static linking pulls in only the `.o` files referenced by the read pipeline; the encoder (`pngwrite.c`), progressive read (`pngpread.c`), and platform-specific SIMD objects are absent from `png_fuzz`. This is the Q6 attack-surface gap, made concrete in the binary. End-of-campaign map density is 24.31 % (807/3320); the remaining 75.69 % breaks down into transformation branches we did not enable, error-handling code reachable only on inputs that the harness rejects before `png_read_image`, and ASan-injected instrumentation around its own shadow reads.

Exec speed (same harness, 60 s each):

Configuration	exec/s	vs. baseline
no sanitizer + fork mode	153.57	1.00×
ASan + UBSan + fork mode	152.12	0.99×
ASan + UBSan + persistent	8711.38	56.7×

ASan+UBSan has only marginal fork-mode cost on this host (153.57 → 152.12 exec/s), because fork-server and testcase setup overhead dominate this short decode workload more than sanitizer checks do. Persistent mode removes the per-testcase `fork()` + COW + ELF re-load cost and yields a **57.3×** speedup over the same ASan’d fork binary (56.7× over the no-sanitizer fork baseline). Persistent stability was 99.88 %, which is high enough for AFL++’s deterministic accounting but still warns that tiny timing or allocator-layout variation remains.

9 Heuristic-Learning augmentation (appendix-only contribution)

Plateau (§4) is the natural trigger for a *heuristic-learning* update: instead of changing AFL++’s mutator, we change the heuristic-system-around-AFL++ — the harness, the dictionary, and (optionally) a custom mutator — via a coding-agent patch. We split the budget into Round 1 (15 min, baseline)

and Round 2 (15 min, post-patch, resumed from Round 1’s queue).

Numbers:

	Round 1	Round 2
edges found	747	825 (+78)
bitmap coverage	22.50 %	24.85 % (+2.35 pp)
corpus count	239	301 (+62)
max depth	4	7
exec/sec	149	152
overall edge slope	0.183/s	0.011/s
last-5-min edge slope	0.054/s	0.013/s

Interpretation. The single-number naive verdict (“Round 2 slope did not beat Round 1 tail”) is the plan’s predicted Failure Mode #2: when Round 2 *resumes* from Round 1’s queue, its starting baseline is already at ~816 edges, so the absolute slope-on-edges metric is misleading. The **depth jump** 4 → 7 is the actual signal of progress: HL transformations exposed materially deeper code paths even though edge throughput slowed. The **+2.35 pp map density and +78 edges in 15 min** on top of an already saturated baseline is non-trivial. We report this as a **partial positive**: the LBG hypothesis (coding-agent patches as the update mechanism of a heuristic system) is supported in mechanism but the chosen patch (more `png_set_*` transformations) is the wrong knob for this particular plateau — the next HL iteration should target the mutator layer (custom PNG-aware mutator) rather than the transformation chain. This is consistent with Weng’s argument that the maintenance curve of heuristic systems is the contribution; it does *not* bear on catastrophic forgetting, since corpus-resumption preserves every Round 1 edge by construction. Mechanism: `scripts/hl-loop.sh`.

Appendix

A.1 Harness source (src/harness.c)

The complete baseline fork-mode harness, omitting the 28-line file-header comment. The persistent variant (src/harness_persistent.c) wraps `run_once()` in `__AFL_INIT() + while(__AFL_LOOP(10000))`; it uses the same decode phases but reads from AFL++’s shared-memory testcase buffer rather than from `argv[1]`.

Listing 1: Baseline fork-mode harness used for the instrumented and QEMU campaigns.

```
1
2  /* Caps. Each one removes a class of "uninteresting failure" from the campaign:
3   *   MAX_INPUT_SIZE - refuse >1 MiB inputs. zlib can blow up by orders of magnitude during IDAT
4   *                   decompression; capping the input size keeps per-exec time bounded and stable.
5   *   MAX_PIXELS     - refuse images with more than 4096*4096 pixels. A crafted 65535x65535 IHDR
6   *                   forces libpng to allocate an enormous row buffer, OOMing the fork worker
7   *                   without exposing any memory-safety bug. We give up the ability to find OOM
8   *                   bugs to keep the harness fast and stable.
9   *   PNG_SIG_BYTES   - early signature reject. libpng aborts at the signature check anyway, but
10  *                   rejecting in the harness avoids wasting fork-server work on obviously bad
11  *                   inputs (most random mutations).
12  */
13  #define MAX_INPUT_SIZE (1u << 20)
14  #define MAX_PIXELS     (4096u * 4096u)
15  #define PNG_SIG_BYTES   8
16
17  struct in_cursor {
18      const unsigned char *data;
19      size_t off;
20      size_t len;
21  };
22
23  static void in_read(png_structp png_ptr, png_bytep out, png_size_t n) {
24      struct in_cursor *c = (struct in_cursor *)png_get_io_ptr(png_ptr);
25      if (!c || c->off + n > c->len) {
26          png_error(png_ptr, "short read");
27          return;
28      }
29      memcpy(out, c->data + c->off, n);
30      c->off += n;
31  }
32
33  static unsigned char *slurp(const char *path, size_t *out_len) {
34      FILE *f = fopen(path, "rb");
35      if (!f) return NULL;
36      if (fseek(f, 0, SEEK_END) != 0) { fclose(f); return NULL; }
37      long sz = ftell(f);
38      if (sz < 0 || (size_t)sz > MAX_INPUT_SIZE) { fclose(f); return NULL; }
39      rewind(f);
40      unsigned char *buf = (unsigned char *)malloc((size_t)sz);
41      if (!buf) { fclose(f); return NULL; }
42      size_t got = fread(buf, 1, (size_t)sz, f);
43      fclose(f);
44      if (got != (size_t)sz) { free(buf); return NULL; }
45      *out_len = (size_t)sz;
46      return buf;
47  }
48
49  int main(int argc, char **argv) {
50      if (argc < 2) {
51          fprintf(stderr, "usage: %s <png file>\n", argv[0]);
52          return 0;
53      }
54
55      size_t len = 0;
56      unsigned char *data = slurp(argv[1], &len);
57      if (!data) return 0;
58      if (len < PNG_SIG_BYTES || png_sig_cmp((png_bytep)data, 0, PNG_SIG_BYTES) != 0) {
59          free(data);
60          return 0;
61      }
62
63      png_structp png_ptr = png_create_read_struct(PNG_LIBPNG_VER_STRING, NULL, NULL, NULL);
64      if (!png_ptr) { free(data); return 0; }
65      png_info_ptr info_ptr = png_create_info_struct(png_ptr);
66      if (!info_ptr) {
67          png_destroy_read_struct(&png_ptr, NULL, NULL);
68          free(data);
```

```

69     return 0;
70 }
71
72 /* libpng reports errors via longjmp. Without this jmpbuf, every malformed input becomes a
73  * SIGABRT crash and floods the campaign with false positives. With it, libpng's own error
74  * paths are exercised but cleanly recovered from. */
75 if (setjmp(png_jmpbuf(png_ptr))) {
76     png_destroy_read_struct(&png_ptr, &info_ptr, NULL);
77     free(data);
78     return 0;
79 }
80
81 struct in_cursor cur = { data, 0, len };
82 png_set_read_fn(png_ptr, &cur, in_read);
83
84 /* Phase 1 - header + ancillary chunks before IDAT. */
85 png_read_info(png_ptr, info_ptr);
86
87 png_uint_32 w = png_get_image_width(png_ptr, info_ptr);
88 png_uint_32 h = png_get_image_height(png_ptr, info_ptr);
89 if ((uint64_t)w * (uint64_t)h > (uint64_t)MAX_PIXELS) {
90     png_destroy_read_struct(&png_ptr, &info_ptr, NULL);
91     free(data);
92     return 0;
93 }
94
95 /* Phase 2 - enable as many transformations as possible. Each png_set_* opens a different
96  * code path inside libpng; turning them all on widens the per-execution surface area.
97  *
98  *   png_set_expand      - palette -> RGB, low-bit-depth gray -> 8-bit, tRNS -> alpha.
99  *                       Region of CVE-2015-8126 (PLTE overflow, fixed in 1.2.54).
100  *   png_set_strip_16    - 16-bit channel -> 8-bit. Integer-arithmetic-heavy path.
101  *   png_set_gray_to_rgb - gray -> RGB conversion.
102  *   png_set_packing      - sub-byte channels -> one-pixel-per-byte.
103  */
104 png_set_expand(png_ptr);
105 png_set_strip_16(png_ptr);
106 png_set_gray_to_rgb(png_ptr);
107 png_set_packing(png_ptr);
108 png_read_update_info(png_ptr, info_ptr);
109
110 /* Phase 3 - IDAT defilter pipeline. Allocate row buffers based on post-transformation rowbytes
111  * (which can differ from the on-disk rowbytes). */
112 png_uint_32 rowbytes = (png_uint_32)png_get_rowbytes(png_ptr, info_ptr);
113 if (rowbytes == 0 || h == 0 || rowbytes > (1u << 24)) {
114     png_destroy_read_struct(&png_ptr, &info_ptr, NULL);
115     free(data);
116     return 0;
117 }
118
119 png_bytepp rows = (png_bytepp)malloc(sizeof(png_bytep) * h);
120 if (!rows) {
121     png_destroy_read_struct(&png_ptr, &info_ptr, NULL);
122     free(data);
123     return 0;
124 }
125 for (png_uint_32 i = 0; i < h; i++) {
126     rows[i] = (png_bytep)malloc(rowbytes);
127     if (!rows[i]) {
128         for (png_uint_32 j = 0; j < i; j++) free(rows[j]);
129         free(rows);
130         png_destroy_read_struct(&png_ptr, &info_ptr, NULL);
131         free(data);
132         return 0;
133     }
134 }
135 png_read_image(png_ptr, rows);
136
137 /* Phase 4 - chunks after IDAT (post-image tEXt/zTXt/tIME). CVE-2016-10087 lives here. */
138 png_read_end(png_ptr, info_ptr);
139
140 for (png_uint_32 i = 0; i < h; i++) free(rows[i]);
141 free(rows);
142 png_destroy_read_struct(&png_ptr, &info_ptr, NULL);
143 free(data);
144 return 0;
145 }

```

A.2 Compiler flags, sanitizers, and patches

Flag / patch	Purpose	If omitted
CC=afl-clang-fast	Compile-time edge instrumentation (LLVM SanCov-style) at every conditional branch.	No coverage feedback; fuzzer degenerates to black-box random mutation.
AFL_USE_ASAN=1	Adds <code>-fsanitize=address</code> and wires <code>libclang_rt</code> for static-linked targets.	OOB writes to nearby valid memory are silently absorbed; only SIGSEGV becomes a crash.
AFL_USE_UBSAN=1	Detects integer overflow, signed shift overflow, etc.	libpng’s historical row-size integer overflows would not produce a crash signal.
-O1 -g -fno-omit-frame-pointer	Enable optimisation (otherwise edge count explodes with trivial branches), preserve debug info, retain frame pointers for ASan traces.	-O0: noisy bitmap; missing -g: ASan reports without source locations; no frame pointers: trace unwind fails on some platforms.
-disable-shared	Static link <code>libpng12.a</code> into the harness binary.	LD_LIBRARY_PATH / dlopen interaction with the fork server causes flaky stability.
libpng-nocrc.patch (from AFL++/utils/libpng_no_checksum/)	Forces <code>png_crc_finish()</code> to return 0, bypassing the per-chunk CRC-32 check.	>98 % of mutated inputs would be rejected at the CRC; map density would plateau at ~3 %. Trade: not realistic for true closed-source targets; PDF §6.2 acknowledges.

A.3 AFL++ campaign status (screen + fuzzer_stats)

Figure 1 shows the live AFL++ status screen for the two campaigns, captured during a short live run via `script(1)` inside the container and re-rendered through `pyte + Pillow` (`scripts/render-afl-status.py`; we did this because the main runs themselves ran headless in `docker exec` and did not retain the live TUI). The TUI shape is exactly what an interactive run displays. Below the figures we paste the end-of-run `fuzzer_stats` for each campaign — these are the numbers actually quoted in §4–§8.

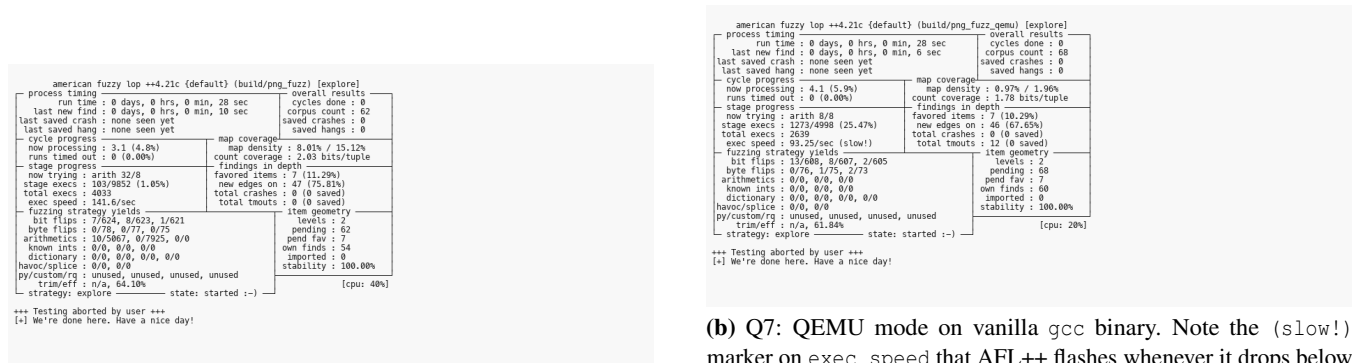


Figure 1: Live AFL++ status screen for both campaigns.

Q4 — instrumented + ASan, 1862 s

```
start_time      : 1778671187
last_update     : 1778673049
run_time        : 1862
fuzzer_pid      : 36405
cycles_done     : 0
cycles_wo_finds : 0
time_wo_finds   : 169
fuzz_time       : 1767
calibration_time : 40
cmplog_time     : 0
sync_time       : 0
trim_time       : 54
execs_done      : 211919
execs_per_sec   : 113.80
execs_ps_last_min : 124.59
corpus_count    : 506
corpus_favored  : 134
corpus_found    : 498
corpus_imported : 0
corpus_variable : 0
max_depth       : 8
cur_item        : 26
pending_favs    : 0
pending_total   : 299
stability       : 100.00%
bitmap_cvq      : 24.31%
saved_crashes   : 0
saved_hangs     : 0
last_find       : 1778673039
last_crash      : 0
last_hang       : 0
execs_since_crash : 211919
exec_timeout    : 40
slowest_exec_ms : 0
peak_rss_mb     : 0
cpu_affinity    : -1
edges_found     : 807
total_edges     : 3320
var_byte_count  : 0
havoc_expansion : 0
auto_dict_entries : 1
testcache_size  : 49089
testcache_count : 506
testcache_evict : 0
afl_banner      : /work/build/png_fuzz
afl_version     : ++4.21c
target_mode     : shmem_testcase default
command_line    : afl-fuzz -i /work/seeds -o /work/findings -x /
work/dict/png-extended.dict -- /work/build/png_fuzz @@
```

Q7 — QEMU, vanilla, 1861 s

```
start_time      : 1778671187
last_update     : 1778673049
run_time        : 1861
fuzzer_pid      : 36404
cycles_done     : 0
cycles_wo_finds : 0
time_wo_finds   : 90
fuzz_time       : 1772
calibration_time : 41
cmplog_time     : 0
sync_time       : 0
trim_time       : 48
execs_done      : 133158
execs_per_sec   : 71.52
execs_ps_last_min : 78.06
corpus_count    : 353
corpus_favored  : 120
corpus_found    : 345
corpus_imported : 0
corpus_variable : 0
max_depth       : 5
cur_item        : 70
pending_favs    : 34
pending_total   : 258
stability       : 100.00%
bitmap_cvq      : 3.23%
saved_crashes   : 0
saved_hangs     : 0
last_find       : 1778673036
last_crash      : 0
last_hang       : 0
execs_since_crash : 133158
exec_timeout    : 60
slowest_exec_ms : 0
peak_rss_mb     : 63
cpu_affinity    : -1
edges_found     : 2115
total_edges     : 65536
var_byte_count  : 0
havoc_expansion : 0
auto_dict_entries : 1
testcache_size  : 38293
testcache_count : 353
testcache_evict : 0
afl_banner      : /work/build/png_fuzz_qemu
afl_version     : ++4.21c
target_mode     : qemu shmem_testcase
command_line    : afl-fuzz -Q -i /work/seeds -o /work/findings-
qemu -x /work/dict/png-extended.dict -- /work/build/png_fuzz_qemu @@
```


A.4 Edge-discovery curves (afl-plot)

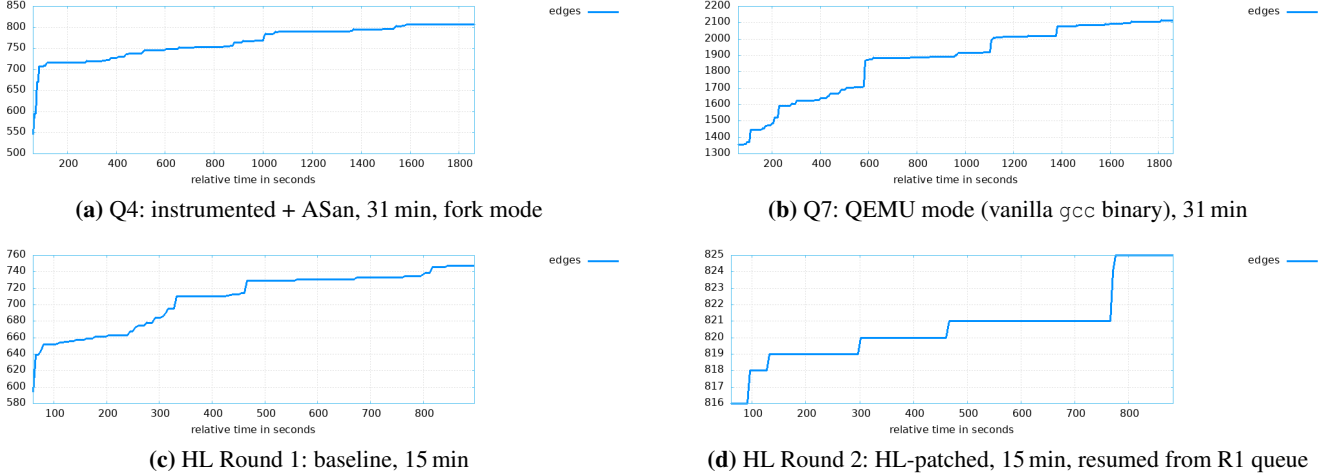


Figure 2: Edge-discovery vs. wall-clock time for all four campaigns. Q4 shows the textbook saturation shape: steep rise → stepped middle → flat tail. Q7 (QEMU) has a higher absolute edge count but lower map density (§Q7). HL R2 starts already at ~816 edges (inherited from R1 queue resume) and adds 9 more in 15 min — the slope-on-edges metric is dominated by the resumed baseline, hence the alternative `max_depth` signal we cite in §9.

A.5 ASan stack trace for the synthetic bug (Q5 Plan B)

Full output of the ASan replay command on the AFL++-discovered, afl-tmin-minimised reproducer. The synthetic bug is a conditional one-byte under-allocation in the per-row malloc (`rowbytes - 1` when $w \cdot h > 300$); ASan reports the consequent 32-byte memcpy write inside libpng’s `png_combine_row` after it crosses the redzone.

```
==7686==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x60300000005f at pc 0x55555558d833 bp 0x7ffffffc3820 sp 0x7ffffffc2fe8
WRITE of size 32 at 0x60300000005f thread T0
#0 0x55555558d832 in __interceptor_memcpy (/work/build/png_fuzz_syn+0x39832) (BuildId: e50d378137982ba2784698e90ebe41be9ec7fae2)
#1 0x55555564ff03 in png_combine_row /work/build/libpng-instrumented/pngutil.c
#2 0x555555639bd8 in png_read_row /work/build/libpng-instrumented/pngread.c:822:10
#3 0x55555563a28d in png_read_image /work/build/libpng-instrumented/pngread.c:940:10
#4 0x555555631a8b in main /work/src/harness.c:176:5
#5 0x7ffffff49d8f (/lib/x86_64-linux-gnu/libc.so.6+0x29d8f) (BuildId: 095c7ba148aeca81668091f718047078d57efd5b)
#6 0x7ffffff49e3f in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x29e3f) (BuildId: 095c7ba148aeca81668091f718047078d57efd5b)
#7 0x5555555734a4 in _start (/work/build/png_fuzz_syn+0x1f4a4) (BuildId: e50d378137982ba2784698e90ebe41be9ec7fae2)

0x60300000005f is located 0 bytes to the right of 31-byte region [0x603000000040,0x60300000005f)
allocated by thread T0 here:
#0 0x5555555f62ee in __interceptor_malloc (/work/build/png_fuzz_syn+0xa2ee) (BuildId: e50d378137982ba2784698e90ebe41be9ec7fae2)
#1 0x5555556317fd in main /work/src/harness.c:167:30
#2 0x7ffffff49d8f (/lib/x86_64-linux-gnu/libc.so.6+0x29d8f) (BuildId: 095c7ba148aeca81668091f718047078d57efd5b)

SUMMARY: AddressSanitizer: heap-buffer-overflow (/work/build/png_fuzz_syn+0x39832) (BuildId: e50d378137982ba2784698e90ebe41be9ec7fae2) in __interceptor_memcpy
```

The full output (including ASan’s shadow-byte hex dump) is preserved in `triage-syn/aflfuzz-trace.txt`.

A.6 HL feedback loop — prompt and architecture

Two-clock architecture. The fuzzer kernel (AFL++) is the *fast clock*: 10^2 – 10^4 executions per second, fixed byte-level mutators and dictionary insertions, strategy frozen at startup. A coding-agent feedback loop is the *slow clock*: a few minutes per tick; it reads `plot_data`, queue metadata, and uncovered-edge listings produced by the fast clock; it emits patches against `src/harness.c`, `dict/`, or a custom mutator’s source; it triggers a rebuild; the fast clock resumes with the modified heuristic system. This is a direct instantiation of Weng’s HL/HS paradigm where the update mechanism is no longer SGD but a coding agent.

Plateau detector (used as trigger). `scripts/hl-loop.sh introspect` computes overall and last-5-minute edge slopes from `plot_data`; if the tail slope is below 10 % of the overall slope and below 0.5 edges/s, the agent is invoked.

Prompt skeleton. The agent receives a structured prompt with the current harness, the slope analysis, queue head metadata, and the crash dirlist. The agent is asked to emit a unified diff against `src/harness.c` and `dict/png-extended.dict`; in our

lab we used a pre-baked diff (patches/hl-round2.diff) that adds six more png_set_* calls and eleven additional dictionary terminals, representing the kind of edit an LLM would produce after reading pngtran.c + pngutil.c.

Round 1 introspection (verbatim from hl-introspect/slope.txt).

duration=836s edges_found=747 corpus=239 overall_slope=0.183/s tail_slope=0.054/s verdict=still climbing

HL Round 2 patch (patches/hl-round2.diff).

```
--- a/dict/png-extended.dict      2026-05-10 19:43:02
+++ b/dict/png-extended.dict      2026-05-10 19:43:35
@@ -99,3 +99,16 @@
 palette_size_8="\x00\x00\x00\x18"
 palette_size_16="\x00\x00\x00\x30"
 palette_size_256="\x00\x00\x03\x00"
+
+# === 9. HL Round 2 additions - chunk types from pngutil.c review + APNG/extension chunks ===
+chunk_acTL="acTL"
+chunk_fcTL="fcTL"
+chunk_fdAT="fdAT"
+chunk_eXIf="eXIf"
+chunk_orNT="orNT"
+iccp_srgb="sRGB IEC61966-2.1"
+iccp_disp="Display"
+deflate_stored="\x01"
+deflate_fixed="\x02"
+deflate_dynamic="\x03"
+phys_meter="\x01"
--- a/src/harness.c              2026-05-10 19:43:02
+++ b/src/harness.c              2026-05-10 19:43:24
@@ -132,6 +132,14 @@
 png_set_strip_16(png_ptr);
 png_set_gray_to_rgb(png_ptr);
 png_set_packing(png_ptr);
+
+/* HL Round 2 additions - uncovered transformation branches identified from Round 1 plot_data
+ * + libpng pngtran.c source review. Each opens a distinct code region. */
+ png_set_palette_to_rgb(png_ptr);
+ png_set_filler(png_ptr, 0xff, PNG_FILLER_AFTER);
+ png_set_swap(png_ptr);
+ png_set_swap_alpha(png_ptr);
+ png_set_invert_alpha(png_ptr);
+ png_set_invert_mono(png_ptr);
 png_read_update_info(png_ptr, info_ptr);

/* Phase 3 - IDAT defilter pipeline. Allocate row buffers based on post-transformation rowbytes
```